AttestFlow

Proof-Gated Cross-Chain Payments for Autonomous Agents

BUIDL CTC 2026 FALL · AI TRACK
EVIDENCE EDITION · SEPTEMBER 2026

> **Natural language expresses intent. Attestcoin establishes truth. A deterministic ASC moves money.**

AttestFlow lets a user state a settlement rule such as:

“When I receive at least 0.01 ETH on Sepolia, release 10% on Creditcoin.”

The agent watches only attested source blocks, builds a real Attestcoin proof, and submits it to an Attestcoin Smart Contract on CC3. The contract verifies the proof, derives the transaction identity from its Merkle path, checks the attested receipt and policy, releases escrow, and publishes a destination-bound result. The exact payload is then executed by a validating Inbox on Sepolia.

Live result at a glance

Public-testnet leg	Transaction
Sepolia client payment · 0.01 ETH	`0x6ac68b…55e7`
CC3 proof-gated settlement · 0.001 CTC released	`0xec29d5…e4c2`
Sepolia execution of the same payload	`0xc692a1…47fb`

62
LIVE CROSS-CHAIN CHECKS

32
AUTOMATED TESTS

2
PUBLIC TESTNETS

0
TRUSTED PAYMENT ASSERTIONS

```
npm ci && npm run judge:verify
# → { "step": "judge-verify", "status": "SUCCESS" }
```

1. Problem and product

Cross-chain payment automation usually asks users to trust an operator, a bridge, or an oracle report. AI agents make the usability problem easier but the truth problem worse: an LLM can interpret intent, yet must never be allowed to hallucinate a payment into existence.

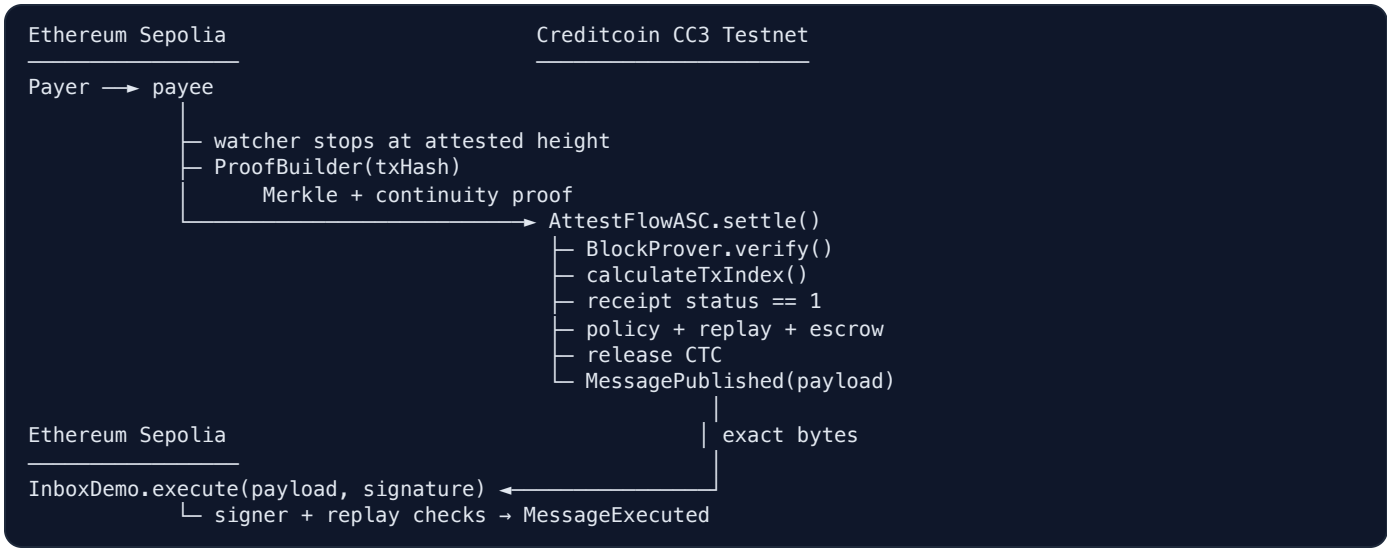
For a cross-border freelancer or an automated treasury, a credible payment agent needs:

1. **source-chain truth** — proof that the exact transaction is in a confirmed chain;
2. **execution truth** — proof that it succeeded and matched the intended recipient and amount;
3. **deterministic settlement** — a fixed beneficiary, ratio, destination, and replay boundary;
4. **closed-loop delivery** — the settlement result must return as verifiable bytes, not a screenshot.

AttestFlow separates these responsibilities:

Layer	Responsibility	Trust model
Intent	Natural-language rule → reviewable policy draft	fail-closed compiler; explicit activation; optional LLM is advisory
Truth	Source inclusion + continuity	Attestcoin Merkle and continuity proofs
Execution	receipt, policy, replay, payout	deterministic CC3 smart contract
Return	destination-bound result	exact CC3-published payload; explicit testnet adapter boundary

Architecture



The agent is autonomous but not authoritative. It can decide **when** to submit work; it cannot make a settlement pass without the proof and on-chain invariants.

User experience

The product dashboard supports rule creation and activation, a four-stage `match → proved → settled → delivered` timeline, natural-language history queries, and a dedicated judge view that reduces the proof trail to one linked story. A deterministic `--tx` mode makes rehearsals repeatable while preserving the full proof and execution path.

2. Attestcoin integration

AttestFlow uses the protocol as the settlement trust root, not as a decorative API call.

Protocol surface	Integration
ChainInfo precompile · `0x0FD3`	Runtime validation that Sepolia `chainId 11155111` maps to `chainKey 1`
Hosted `ProofBuilder`	Real transaction Merkle proof + source-chain continuity proof
Shared batch proof	`getBatchProof()` + native `verifyBatch()` gate 2–10 backlog payments
BlockProver · `0x0FD2`	Synchronous `verify()` inside every settlement
`calculateTxIndex()`	Canonical proof identity and replay-key binding
Encoding v1	On-chain decode of source fields and receipt status
Writability interface	destination-bound publish → sign → deliver → validate adapter

Why inclusion is not enough

The BlockProver establishes that the encoded transaction belongs to an attested source block. It does **not** establish that execution succeeded or that the payment matches a business rule. The ASC therefore decodes the last encoding-v1 chunk and requires receipt `status == 1`.

The full settlement gate is:

1. policy exists, is active, and matches `chainKey`;
2. BlockProver accepts Merkle and continuity proofs;
3. `calculateTxIndex(merkleProof)` equals the supplied source index;
4. proof-derived `sourceTxId` is unused;
5. decoded receipt status is successful;
6. native target/value or ERC-20 target/calldata matches the stored payee and threshold;
7. escrow covers the deterministic payout;
8. settlement publishes to the policy-fixed destination.

A replay vulnerability closed during hardening

`verify()` does not receive a transaction index. An earlier design used the operator-supplied index in `keccak(chainKey, height, txIndex)`; a malicious operator could reuse a valid proof with a different index and obtain a new replay key. The deployed v2 contract derives the index from Merkle sibling directions and requires equality before reserving the replay key.

Encoding and payment support

```
txBytes = abi.encode(uint8 txType, bytes[] chunks)

chunks[0]  = nonce · gasLimit · from · toIsNull · to · value · data
chunks[last] = receiptStatus · gasUsed · logs · bloom
```

The contract supports native transfers plus ERC-20 `transfer` and `transferFrom`. Token discovery first filters indexed recipient events, then requires the transaction calldata recipient and amount to agree. This avoids settling unrelated vault activity that merely emitted a `Transfer` event.

Empirical protocol finding

On CC3 Testnet, EOA `verifyAndEmitSingle` succeeded while contract-context `verifyAndEmit()` reverted. `verify()` succeeds from the ASC, so AttestFlow uses it synchronously and emits domain events itself. The cryptographic proof gate remains identical; only a redundant precompile event is omitted.

3. Security, resilience, and evidence

Threat model

Actor / failure	Control
agent fabricates a payment	impossible without a BlockProver-accepted proof
source transaction reverted	attested receipt `status == 1` required
operator changes transaction index	proof-derived index equality check
valid proof submitted twice	proof-derived `sourceTxId` replay guard
agent redirects payout or message	beneficiary and destination stored in policy
forged token event	token address, selector, recipient, and amount checked from calldata
destination payload replay	`executedPayloads[payloadHash]` guard
one invalid payment in a backlog	shared batch proof must pass before any member enters settlement
RPC timeout between chains	checkpoint CC3 leg; recover original payload from receipt; finish only missing leg
LLM emits unsafe or incomplete values	no money defaults; schema rejection; inactive draft; explicit activation

The owner controls policy creation, operators, and escrow. That governance trust is explicit: Attestcoin authenticates source facts; it does not decide whether an owner-created business policy is economically sensible.

Cross-chain failure recovery

The two chains cannot commit atomically. AttestFlow persists the CC3 result before destination I/O. On retry it checks `settledTxs(sourceTxId)`, finds `PaymentSettled`, re-reads the receipt, extracts the exact `MessagePublished` bytes, and checks the destination replay map. The recorded v2 run encountered a Sepolia RPC timeout after CC3 finalized and recovered without a second settlement.

Evidence verifier

The repository does not ask judges to trust log screenshots. `npm run verify:evidence` independently re-reads both networks and validates source fields plus the proof-derived `sourceTxId`; CC3 settlement and destination execution status; the three contract events; payload-hash equality; both replay guards; and deployed runtime bytecode against local compiled artifacts.

The unified `judge:verify` adds two public Sourcify creation/runtime exact-match lookups, a fresh shared proof spanning three attested blocks with native `verifyBatch()`, and a fresh single proof with `verifySingle()`.

Current payload integrity anchor:

```
0x4845f5ca486987ddb30d486e58f36ed0cebbf5e514d20783d220b06f0d523faa
```

Engineering acceptance

32 automated tests: 15 contract tests cover native/ERC-20 paths, receipt/proof/index failures, replay, policy, escrow, authorization, and destination; 11 agent tests cover protocol decoding, calldata, Merkle identity, policy reuse, adversarial model JSON, and drafts; 6 web tests cover exact money parsing, fail-closed rules, bounds, and activation.

Delivery gates: TypeScript, all tests, production build, dependency audit, Dependabot, and CodeQL; zero known production vulnerabilities. Protocol acceptance adds live single and multi-block batch proofs, 62 cross-chain checks, and two Sourcify exact matches.

4. AI role, limitations, and path to production

AI where it helps; determinism where money moves

AttestFlow uses natural language to reduce policy-authoring friction and to query settlement history. An OpenAI-compatible model can be enabled and is disclosed, but the default local parser keeps the demo reproducible. Either way, AI output is compiled into explicit fields:

- source chain and asset;
- exact payee;
- minimum amount;
- payout ratio;
- fixed beneficiary and destination.

Compilation is not authorization. Before any optional model call, the local compiler requires one self-receipt clause that binds the agent wallet, Sepolia source, and exactly one supported asset/amount, plus exactly one payout percentage. Strict model JSON must agree with those locally extracted values. The dashboard stores every compiled rule as an inactive draft, shows the exact money fields, and revalidates them before explicit activation. Model-assisted CLI output similarly remains `REVIEW_REQUIRED` until a later `--activate <rule-id>` command. Even after activation, no model output can bypass the proof, receipt, replay, policy, or escrow checks. This is the core product principle: **agents propose; users authorize policy; cryptography establishes truth; code moves money.**

Honest Writability boundary

The official Writability Outbox/Inbox contracts are not available on the target testnet at submission time. AttestFlow therefore implements the documented four-stage interface with `MessagePublished`, one authorized EIP-191 relayer, and `InboxDemo`.

This demonstrates payload binding, delivery, validation, replay protection, and crash recovery. It does **not** claim attestor-quorum security. Migrating to the official stack replaces the signing/validation adapter while retaining the policy-bound payload and recovery state machine.

Why this can become a product

The freelancer demo is one instance of a reusable primitive:

- marketplace escrow released by externally verified payment;
- proof-gated invoice factoring;
- treasury automation across supported source chains;
- agent-to-agent commerce with cryptographic receipts;
- compliance workflows that act only on verified external events.

The ChainInfo registry makes additional sources discoverable, while the ASC keeps the money path deterministic and auditable.

Next milestones

1. swap the adapter for official Writability quorum validation when deployed;
2. add per-policy spending caps, timelocks, and multisig ownership;
3. expand to additional ChainInfo-supported source chains;
4. formalize the encoding decoder and fuzz proof-bound policy matching;
5. expose human-readable risk explanations without granting AI settlement authority.

Conclusion

AttestFlow demonstrates a concrete design for autonomous finance: natural language supplies intent, Attestcoin supplies cross-chain truth, and a deterministic ASC supplies enforceable outcomes. The claim is not a slide — it is a public payment, a proof-gated CC3 settlement, an integrity-bound return message, and a verifier anyone can rerun.

Repository: github.com/ximilu0114-droid/ximilu-hackson
Judge view: `http://localhost:3100/judge`
Technical mapping: `docs/integration.md`